C# & .NET Basic Certification Exam — Paper A

Focus: .NET Environment, Visual Studio, Variables, Operators & Control Flow

Course: Introduction to C# Programming	**Duration:** 60 Minutes
Topic: Basic .NET & C# Fundamentals	**Total Marks:** 50 Marks
Instructions: Write all code legibly. Answer all questions in Sections A, B, and C.	**Passing Grade:** 60%

Section A: .NET Environment & Core C# Syntax (15 Marks)

1. (3 Marks) Which component of the .NET Framework is responsible for executing C# applications, managing memory, and providing garbage collection?

A) Visual Studio Community IDE
B) Common Language Runtime (CLR)
C) Just-In-Time (JIT) Compiler only
D) C# Source Code Editor

2. (3 Marks) What is the function of the Just-In-Time (JIT) compiler in the .NET Framework execution pipeline?

A) Converts C# source code directly into HTML and CSS web files.
B) Compiles C# source code into C++ native code before installation.
C) Converts Intermediate Language (IL) code into native machine code at runtime.
D) Checks C# code for syntax errors inside Microsoft Visual Studio.

3. (4 Marks) Consider the variable declarations below. Which line contains a compile-time error, and why?

```
int age = 25;
double rating = 4.8;
int total = rating; // Line 3
```

Explain how to fix Line 3 using explicit casting in C#.

4. (5 Marks) Briefly describe the role of Microsoft Visual Studio Community in C# application development. List two key features provided by the IDE.

Answer lines:

Section B: Conditional Statements & Logical Operators (15 Marks)

5. (5 Marks) What will be printed to the Console when the following C# snippet executes?

```
int x = 10;
int y = 20;
if (x > 5 && y < 15) {
    Console.WriteLine("Option A");
} else if (x == 10 || y == 10) {
    Console.WriteLine("Option B");
} else {
    Console.WriteLine("Option C");
}
```

6. (5 Marks) Write a short C# statement to read user input from the console and convert it safely into an integer named 'userNumber'.

Write your C# code below:

7. (5 Marks) Compare a 'switch' statement with an 'if-else if' ladder in C#. State one scenario where using a 'switch' statement is preferred.

Section C: Loops & Practical Problem Solving (20 Marks)

8. (8 Marks) Determine the exact console output of the following loop:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    Console.Write(i + " ");
}
```

9. (12 Marks) Coding Task: Write a complete C# console snippet that uses a 'while' loop to repeatedly ask the user to enter integers until they enter 0. The program should then print the sum of all entered numbers.

Write your C# solution below:

Exam Paper A — Solutions & Marking Scheme

Question 1 Answer: B

The Common Language Runtime (CLR) is the core execution engine of .NET that manages application execution, memory allocation, and garbage collection.

Question 2 Answer: C

The JIT compiler converts Intermediate Language (IL) bytecode into native CPU instructions right before execution on the host machine.

Question 3 Answer: Line 3 fails compile-time check.

Cannot implicitly convert type 'double' to 'int'. Fix: int total = (int)rating; (explicit narrowing conversion).

Question 4 Answer: Visual Studio IDE

Visual Studio Community is a free, extensible IDE for building .NET apps. Features: Code IntelliSense/Auto-completion, integrated Debugger, and Solution Explorer.

Question 5 Answer: Option B

Explanation: (x > 5 && y < 15) is (True && False) = False. Next condition: (x == 10 || y == 10) is (True || False) = True. Thus, 'Option B' prints.

Question 6 Answer: Console Parsing Code

string input = Console.ReadLine(); int userNumber = int.Parse(input); // or int.TryParse(input, out userNumber);

Question 7 Answer: Switch vs If-Else

A switch statement tests a single variable against discrete constant values. It is cleaner and more readable when checking multiple fixed values (e.g. enum or menu choices) compared to repetitive if-else statements.

Question 8 Answer: Output is '1 2 4 5 '

The continue statement skips iteration when i == 3, so 3 is not printed.

Question 9 Answer: While Loop Summing Code

int sum = 0; int number = -1; while (number != 0) { Console.Write("Enter a number (0 to stop): "); number = int.Parse(Console.ReadLine()); sum += number; } Console.WriteLine("Total sum: " + sum);

C# & .NET Basic Certification Exam — Paper B

Focus: Arrays, Lists, Memory Allocation (Stack vs Heap) & OOP Fundamentals

Course: Introduction to C# Programming	**Duration:** 60 Minutes
Topic: Basic .NET & C# Fundamentals	**Total Marks:** 50 Marks
Instructions: Write all code legibly. Answer all questions in Sections A, B, and C.	**Passing Grade:** 60%

Section A: Arrays, Lists & Data Collections (15 Marks)

1. (3 Marks) What is the fundamental difference between an array (`int[]`) and a List (`List`) in C#?

A) Arrays can hold strings, while Lists can only hold numbers.
B) Arrays have a fixed size upon creation, whereas Lists are dynamic in size.
C) Arrays store elements on the Heap, whereas Lists store elements on the Stack.
D) Lists do not allow element access via index `list[i]`.

2. (4 Marks) Predict the output of the following C# array processing code:

```
int[] numbers = { 2, 4, 6, 8 };
int sum = 0;
foreach (int num in numbers) {
    sum += num;
}
Console.WriteLine("Average: " + (sum / numbers.Length));
```

3. (8 Marks) Write C# code snippets to: a) Declare and instantiate a dynamic `List` named 'students'. b) Add two names ("Alice" and "Bob") to the list. c) Print the total count of items in the list.

Write your solution below:

Section B: Memory Management: Stack vs. Heap (15 Marks)

4. (5 Marks) In C#, where are value types (such as `int`, `double`, `bool`) typically allocated versus reference types (such as class instances and `string`)?

5. (10 Marks) Analyze the code snippet below and state where each variable and its data are allocated in memory (Stack or Heap):

```
int score = 95;
string studentName = "Maria";
```

Explanation of memory allocation:

Section C: Fundamentals of Object-Oriented Programming (20 Marks)

6. (5 Marks) Define the terms 'Class' and 'Object' in C#, highlighting the relationship between them.

7. (15 Marks) Practical OOP Challenge: Create a C# class named 'Student' with: • Two private fields: `string name` and `int id`. • A constructor that accepts `name` and `id` to initialize the fields. • A public method `DisplayInfo()` that prints the student's name and ID to the console.

Write the complete class definition below:

Exam Paper B — Solutions & Marking Scheme

Question 1 Answer: B

Arrays have a fixed size defined at allocation. Lists (`List`) are dynamic data structures that automatically resize as elements are added or removed.

Question 2 Answer: 'Average: 5'

Explanation: sum = 2 + 4 + 6 + 8 = 20. Array length = 4. Integer division: 20 / 4 = 5.

Question 3 Answer: List Operations Code

List students = new List(); students.Add("Alice"); students.Add("Bob"); Console.WriteLine(students.Count);

Question 4 Answer: Stack vs Heap Allocation

Value types (int, double, bool, struct) are stored directly on the Stack (or inline inside containing objects). Reference types (class objects, strings, arrays) store object references on the Stack while their actual object content resides on the Heap.

Question 5 Answer: Memory Allocation Analysis

• 'score' is a primitive value type (int) stored directly on the Stack holding value 95. • 'studentName' is a reference variable stored on the Stack containing a pointer/reference to the string object "Maria" located on the Heap.

Question 6 Answer: Class vs Object

A Class is a blueprint or template defining fields, properties, and methods. An Object is an instance created from that blueprint residing in memory.

Question 7 Answer: Student Class Solution

public class Student { private string name; private int id; public Student(string name, int id) { this.name = name; this.id = id; } public void DisplayInfo() { Console.WriteLine($"ID: {id}, Name: {name}"); } }

C# & .NET Exam Paper 1: Core Fundamentals & Control Flow

Comprehensive Test covering .NET Architecture, IDE, Variables, Operators, and Loops

Duration: *90 Minutes* | ***Total Marks:*** *70 Marks* | ***Format:*** *Written & Practical Coding*

Section A: .NET Architecture & Visual Studio IDE (15 Marks)

Test your fundamental knowledge of the runtime, execution model, and development environment.

Q1. What is the primary role of the Common Language Runtime (CLR) in the .NET Framework? [3 Marks]

A) To edit C# source code files

B) To manage memory, execute C# code, and handle thread management

C) To store project files in Visual Studio

D) To convert HTML pages into C# classes

Q2. Explain the difference between Source Code, Intermediate Language (IL/MSIL), and Native Machine Code during C# compilation. [6 Marks]

Q3. In Visual Studio Community, what is the purpose of the Solution Explorer window and the Build solution command? [6 Marks]

Section B: Variables, Data Types & Operators (20 Marks)

Focuses on type safety, implicit/explicit casting, and mathematical/logical operators.

Q4. Which of the following data types is a C# value type stored directly on the Stack? [3 Marks]

A) string

B) int

C) List

D) string[]

Q5. Analyze the following C# code snippet. What will be printed to the Console? [7 Marks]

```
int a = 15;
int b = 4;
double result1 = a / b;
double result2 = (double)a / b;
Console.WriteLine($"{result1} | {result2}");
```

Q6. Write a short C# program that prompts the user to enter a temperature in Celsius via Console.ReadLine(), converts the input string to a double, calculates Fahrenheit (F = C * 9/5 + 32), and displays the formatted result. [10 Marks]

Section C: Control Flow & Iteration (35 Marks)

Evaluates conditional decision making and loop structures.

Q7. Convert the following if-else block into an equivalent switch statement: [10 Marks]

```
if (grade == "A") {
Console.WriteLine("Excellent");
} else if (grade == "B") {
Console.WriteLine("Good");
} else {
Console.WriteLine("Needs Improvement");
}
```

Q8. Write a C# program using a while loop that continuously prompts the user to enter numbers until they type -1. The program should sum all positive numbers entered and output the final sum. [25 Marks]

EXAM ANSWER KEY & MARKING SCHEME

Comprehensive Solutions & Grading Criteria

Solutions: Section A: .NET Architecture & Visual Studio IDE (15 Marks)

Q1. Solution & Criteria:

Correct Answer: B. The CLR is the execution engine of .NET that handles garbage collection, JIT compilation, memory management, and security enforcement.

Q2. Solution & Criteria:

Source code (.cs) is written by developers. The C# compiler (csc) converts source code into Intermediate Language (IL/CIL) stored in assemblies (.dll/.exe). At runtime, the JIT (Just-In-Time) compiler converts IL into CPU-specific native machine code.

Q3. Solution & Criteria:

Solution Explorer displays the hierarchical structure of projects, source code files, dependencies, and references. The Build solution command compiles all modified source files into IL binaries to detect syntax and reference errors.

Solutions: Section B: Variables, Data Types & Operators (20 Marks)

Q4. Solution & Criteria:

Correct Answer: B. Primitive numeric types like int, double, bool, and char are value types stored on the stack.

Q5. Solution & Criteria:

Output: 3 | 3.75
Explanation: a / b performs integer division resulting in 3. (double)a casts 15 to 15.0, forcing floating-point division resulting in 3.75.

Q6. Solution & Criteria:

```
Console.Write("Enter Celsius: ");
string input = Console.ReadLine();
double celsius = Convert.ToDouble(input);
double fahrenheit = (celsius * 9.0 / 5.0) + 32;
Console.WriteLine($"Fahrenheit: {fahrenheit:F2}");
```

Solutions: Section C: Control Flow & Iteration (35 Marks)

Q7. Solution & Criteria:

```
switch (grade) {
case "A":
Console.WriteLine("Excellent");
break;
case "B":
Console.WriteLine("Good");
break;
default:
Console.WriteLine("Needs Improvement");
break;
}
```

Q8. Solution & Criteria:

```
int totalSum = 0;
while (true) {
Console.Write("Enter a number (-1 to stop): ");
int num = int.Parse(Console.ReadLine());
if (num == -1) break;
if (num > 0) totalSum += num;
}
Console.WriteLine($"Total Sum of Positive Numbers: {totalSum}");
```

C# & .NET Exam Paper 2: Collections, Memory & File I/O

Comprehensive Test covering Arrays, Lists, Stack vs Heap, and System.IO File Operations

Duration: *90 Minutes* | ***Total Marks:*** *70 Marks* | ***Format:*** *Written & Practical Coding*

Section A: Arrays & List Collections (20 Marks)

Assess data structures including fixed-size arrays and dynamic List collections.

Q1. What happens if you attempt to access numbers[5] on an array defined as int[] numbers = new int[5];? [4 Marks]

A) It returns 0

B) It automatically expands the array

C) Throws an IndexOutOfRangeException at runtime

D) Throws a compilation error

Q2. Compare C# fixed-size arrays (int[]) with dynamic generic lists (List). Highlight differences in size, methods, and syntax. [6 Marks]

Q3. Write a C# method that accepts a List of student test scores and returns the average score. If the list is empty, return 0.0. [10 Marks]

Section B: Memory Architecture — Stack vs. Heap (20 Marks)

Examines execution frames, memory allocation, value types vs reference types.

Q4. Explain how memory allocation differs when executing int x = 10; versus int[] arr = new int[10];. [10 Marks]

Q5. Analyze the following code. What are the values of listA[0] and listB[0] at the end of execution? Explain why. [10 Marks]

```
List<string> listA = new List<string> { "Apple" };
List<string> listB = listA;
listB[0] = "Banana";
Console.WriteLine(listA[0]);
```

Section C: File Handling & Input/Output (30 Marks)

Focuses on working with text files using the System.IO namespace.

Q6. Identify two advantages of using StreamReader and StreamWriter over File.ReadAllText and File.WriteAllText. [6 Marks]

Q7. Write a complete C# program that reads names from "input_students.txt" line by line, converts each name to UPPERCASE, and writes the uppercase names to "output_students.txt". Include basic exception handling using try-catch. [24 Marks]

EXAM ANSWER KEY & MARKING SCHEME

Comprehensive Solutions & Grading Criteria

Solutions: Section A: Arrays & List Collections (20 Marks)

Q1. Solution & Criteria:

Correct Answer: C. C# arrays are zero-indexed (valid indices 0 to 4). Accessing index 5 triggers an IndexOutOfRangeException.

Q2. Solution & Criteria:

Arrays have a fixed size declared at creation and use .Length. List<T> is dynamic in size, provides methods like .Add(), .Remove(), .Contains(), and uses .Count to inspect item quantity.

Q3. Solution & Criteria:

```
public static double CalculateAverage(List<double> scores) {
if (scores == null || scores.Count == 0) return 0.0;
double sum = 0;
foreach (double score in scores) {
sum += score;
}
return sum / scores.Count;
}
```

Solutions: Section B: Memory Architecture — Stack vs. Heap (20 Marks)

Q4. Solution & Criteria:

int x = 10 allocates a value type variable directly on the stack frame. int[] arr allocates the reference variable arr on the stack, which points to the array object instantiated on the heap.

Q5. Solution & Criteria:

Output: "Banana"
Explanation: List<T> is a reference type. Assigning listB = listA copies the reference pointer, not the object. Both variables point to the same memory location on the heap.

Solutions: Section C: File Handling & Input/Output (30 Marks)

Q6. Solution & Criteria:

StreamReader/Writer stream data line-by-line rather than loading entire multi-gigabyte files into RAM at once, preventing OutOfMemoryException and improving I/O throughput.

Q7. Solution & Criteria:

```
using System;
using System.IO;

class FileProcessor {
static void Main() {
try {
string[] lines = File.ReadAllLines("input_students.txt");
using (StreamWriter writer = new StreamWriter("output_students.txt")) {
foreach (string line in lines) {
writer.WriteLine(line.ToUpper());
}
}
Console.WriteLine("File processing complete.");
} catch (FileNotFoundException) {
Console.WriteLine("Error: Input file not found.");
} catch (Exception ex) {
Console.WriteLine($"Error: {ex.Message}");
}
}
}
```

C# & .NET Exam Paper 3: OOP & Integrated Software Systems

Comprehensive Test covering Classes, Constructors, Methods, Encapsulation, and System Integration

Duration: *90 Minutes* | ***Total Marks:*** *70 Marks* | ***Format:*** *Written & Practical Coding*

Section A: Object-Oriented Fundamentals & Class Design (20 Marks)

Demonstrate knowledge of OOP encapsulation, fields, properties, and access modifiers.

Q1. Which access modifier restricts access to members so they are only visible within the containing class? [3 Marks]

A) public

B) private

C) protected

D) internal

Q2. Explain why backing fields should be private while expose auto-properties with { get; set; } in C#. [7 Marks]

Q3. Write a C# class named Book with private fields for title, author, and price. Provide a constructor, public auto-properties, and a PrintDetails() method. [10 Marks]

Section B: Constructors & Instance Methods (20 Marks)

Evaluates object instantiation, instance method design, and object state validation.

Q4. Design a BankAccount class with a Balance property. Ensure that the Deposit(double amount) method only accepts positive values, and Withdraw(double amount) prevents overdrawing. [20 Marks]

Section C: End-to-End System Integration Project (30 Marks)

Combines Classes, File handling (System.IO), and List collections in a single mini-application.

Q5. Write a complete C# application that reads product records from "products.txt" (format: Name,Price), creates Product objects, stores them in a List, filters products priced over $50.00, and prints the filtered list to the Console. [30 Marks]

EXAM ANSWER KEY & MARKING SCHEME

Comprehensive Solutions & Grading Criteria

Solutions: Section A: Object-Oriented Fundamentals & Class Design (20 Marks)

Q1. Solution & Criteria:

Correct Answer: B. private hides members from external classes, enforcing encapsulation.

Q2. Solution & Criteria:

Private backing fields prevent unauthorized external tampering. Auto-properties maintain encapsulation, allow validation logic inside getters/setters, and enable data-binding frameworks.

Q3. Solution & Criteria:

```
public class Book {
public string Title { get; set; }
public string Author { get; set; }
public double Price { get; set; }

public Book(string title, string author, double price) {
Title = title;
Author = author;
Price = price;
}

public void PrintDetails() {
Console.WriteLine($"'{Title}' by {Author} - ${Price:F2}");
}
}
```

Solutions: Section B: Constructors & Instance Methods (20 Marks)

Q4. Solution & Criteria:

```
public class BankAccount {
public string AccountNumber { get; }
public double Balance { get; private set; }

public BankAccount(string accNum, double initialBalance) {
AccountNumber = accNum;
Balance = initialBalance > 0 ? initialBalance : 0;
}

public void Deposit(double amount) {
if (amount > 0) Balance += amount;
}

public bool Withdraw(double amount) {
if (amount > 0 && amount <= Balance) {
Balance -= amount;
return true;
}
return false;
}
}
```

Solutions: Section C: End-to-End System Integration Project (30 Marks)

Q5. Solution & Criteria:

```
using System;
using System.Collections.Generic;
using System.IO;

class Product {
public string Name { get; set; }
public double Price { get; set; }
public Product(string name, double price) {
Name = name;
Price = price;
}
}

class Program {
static void Main() {
List<Product> productList = new List<Product>();
if (!File.Exists("products.txt")) return;

string[] lines = File.ReadAllLines("products.txt");
foreach (string line in lines) {
string[] parts = line.Split(',');
if (parts.Length == 2) {
string name = parts[0].Trim();
if (double.TryParse(parts[1].Trim(), out double price)) {
productList.Add(new Product(name, price));
}
}
}

Console.WriteLine("--- Products Over $50 ---");
foreach (Product p in productList) {
if (p.Price > 50.00) {
Console.WriteLine($"{p.Name}: ${p.Price:F2}");
}
}
}
}
```